Making a Text Based Turn Based Battle Game in C#

by Kelin Levine

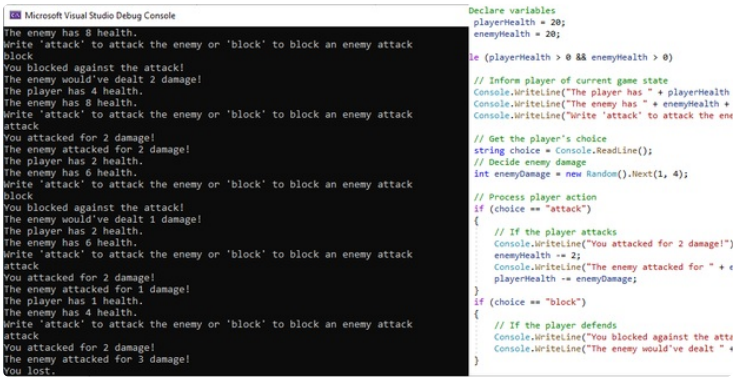
It's easy to take computer graphics and sound for granted if you don't program. Graphics and sound tend to be one of the trickiest parts of modern-day coding, as computers today are all built with different components that work in different ways. That's why you see a lot of programs that make use of a console (that black box usually with white or green text) since it's often much easier to put all of your user interaction in a text box that comes standard with any operating system.
There's also just an inherent beauty in having a whole game or program in nothing but text while maintaining all functionality and keeping it understandable. That's why in this instructable we'll be taking a step back and programming a console window to play a common but long-standing element from any role-playing game: turn-based battles.

This guide contains examples above each step of how the script should look after the step. I strongly recommend reading the step before checking these images as understanding what each part does is more important than having a final product.

This guide is also written for people just coming out of my last instructable. If you've done some outside learning, feel free to skip any bits that you already know. If you think that you're missing something, then you can go back to the first guide at any time. Please keep in mind that many programming concepts are difficult to explain in writing but are simple once you understand them.

Supplies:

A desktop/laptop computer
Any amount of experience with C# (see my previous guide to get started)
An IDE (Visual Studio Community, Visual Studio Code, etc.)



Step 1: Variables and Numbers

Last time we got the very bare basics down. Part of it is that we had the computer remember some text that the user inputted by using a line of code starting with mention of a "string" as well as a name you wrote. I never brought up what this concept is called in that guide, but now that you've (hopefully) had some experience working with them, it's time to move up in the world. This part is incredibly important for any computer programming, so if you don't understand any part of it, please be sure to read it over again.

Those are called *variables*. Variables remember some form of data so that the computer can use it later. The type of data a variable contains (text, number, etc.) depends on the first word in the line of code.

If the first word in any line of code matches a specific keyword that relates to a type of data, then it essentially tells the program "Hey, we're talking about a new variable. This is the data it'll give us, and after the next space is the name". Though the name can technically be anything that doesn't contain either a space or a special character, it's recommended to write out the names of variables in camel-case, which is when you capitalize the first letter of every word except for the first one. For now, only use camel-case when naming *your* variables as there are different ways to format names and explaining all of them would be excessive. Last time, the type of our variable (technically called "return type" because it's the type of data that returns to the computer) was "string". As I said, this means a *string* of letters, which is text. We'll be getting into numbers in this step.

```
int testNumber = 5;
string testString = "test";
```

An integer is a whole number. Integers are often used in programming because whole numbers are simple and what computers can understand the best. To create a variable that holds an integer, begin the line with the keyword "int". For example, if you wanted to create a variable named "myNumber" that contained the number 6, you would write:

int myNumber = 6;

It's the exact same syntax as before: type first, name second, equals sign third, value fourth, and semicolon fifth. You can see more examples (including setting a specific text to a variable) in the images above.

One last thing to note: using a return type when referring to a variable that already exists will break the script if you've already declared to variable in that section of code, but if the same section of code is run twice, then it'll just ignore the other value of the variable.

Step 2: Changing Variables

The most important part of variables is the fact that they're, well, variable. They can be changed while the script is running so that the same code can give different results depending on other factors.

The syntax for changing a variable is identical to making a new one, but without specifying a type. For example, if we wanted to take the variable called "myNumber" that we already created (or *declared*) in the last step and change its value to the number 4, the line of code to do that would be the following:

myNumber = 4;

Note how similar this is to declaring it with the value 6, which is:

int myNumber = 6;

So what's the difference? As I said in the last step, starting the line of code with a return type (like that *int* keyword the example above begins with) tells the computer that you're making (or declaring) a *new* variable. If the line starts with a name, the computer will look for a variable with a matching name and use that to perform whatever code follows it.

```
int testNumber = 5;
string testString = "test";

testNumber = 3;
testString = "new test string";
```

Step 3: Math

Scary concept, I know?

Let's not waste any time. You can use math in place of any number. Here's how you would change the already existing myNumber to 7 using math:

myNumber = 5 + 2;

5 + 2 = 7, thus myNumber will equal 7. You can also use variables in place of numbers. Here's a short example of a script that does the exact same as above, but using more variables:

int firstNumber = 5;

int secondNumber = 2;

int finalNumber = firstNumber + secondNumber;

This results in finalNumber equaling to 7. You can use the operations +, -, *, and / to perform these equations. + is for addition, - is for subtraction, * is for multiplication, and / is for division.

Lastly, if you would like to change a variable containing a number by applying one of the operations above to itself, you would change it the same way as normal but with the operation right before the equals sign. For example, if we wanted to set myNumber to itself multiplied by 2, we would do this:

myNumber *= 2;

That above is identical to:

myNumber = myNumber * 2;

You can also use the variable name followed by either '++' or '--' to add or subtract 1 from them.

myNumber++;

adds 1 to myNumber,

myNumber--;

removes 1 from myNumber.

You can only do this to variables that already exist.

```
int testNumber;
int otherNumber = 4;

testNumber = 3 + 5;
testNumber += 2;
testNumber *= 3;
testNumber /= otherNumber;
```

Step 4: Displaying Player and Enemy Health

Now it's time to begin on the actual game. Go ahead and create a new project so that you have a space to work in. This will be a console application using C# and .NET Core.

We'll start by producing something that displays the player's health and the enemy's health. But before we show anything, we'll have to tell the computer what these two things are.

The health is going to be a number that changes when the player/enemy takes damage. Thus, we want it to be a integer variable. We'll do this by first declaring both of the health variables at the top of our code. You usually want to declare variables at the start of your code, since if a part of your code tries to use a variable that hasn't been declared yet, it'll crash. We do that with this:

int playerHealth = 20;

int enemyHealth = 20;

```
5   class Program
6   {
        0 references
7       static void Main(string[] args)
8       {
9           int playerHealth = 20;
10          int enemyHealth = 20;
11
12          Console.WriteLine("The player has " + playerHealth + " health.");
13          Console.WriteLine("The enemy has " + enemyHealth + " health.");
14      }
15  }
16 }
17
```

This starts both the player's and the enemy's health at 20. Now we just repeat this health back to the user using Console.WriteLine():

Console.WriteLine("The player has " + playerHealth + " health.");

Console.WriteLine("The enemy has " + enemyHealth + " health.");

Now, if you test the program, it will tell you:

The player has 20 health.

The enemy has 20 health.

You can check the image above for help. This is your reminder to save your script, which you should be doing whenever you finish a change anyway.

Step 5: Gameplay Loop

The gameplay for our little game will consist of the player choosing to either attack or block the enemy's attack. This repeats until either the player or the enemy has been defeated. That means all of our code will need to keep repeating until a certain requirement has been met. We will do this with what's called a *while* loop. It will repeat a certain section of code *while* a certain condition is met. This step will be a little bit of a doozy, so hang in there. We do this in 3 parts:

1. We start a line with the word "while"

2. We place our conditions in parenthesis in the same line (more on that later)

3. We put a opening curly bracket in the next line, an indent in the following one, and a closing curly bracket in the one after that

Now we move all of our code that we want to repeat (so everything except for the variables) to in between those curly brackets. If you do it right, there will be a red line. The red line is there because we have yet to tell the program when it should repeat the code.

To fix it, we'll use the variables. We want our code to stop once the player *or* enemy runs out of health. Since we tell the computer when it *should* be running, we tell our code to run as long as the player and enemy both have more than 0 health.

The way that the computer checks things is that you have to give it some kind of condition that will always either be correct or incorrect. For example, 1 > 0 is correct, so if that's the condition in our while loop, then the computer will always run our code. However, 1 < 0 is incorrect, so if that's our condition, then the computer will never run our code.

Keep in mind what I said earlier about using numeric variables in place of numbers. You can do that here. If we have a variable in a condition, then we can control if the condition is or isn't met while the script is running. For example, the while loop in this:

```
int runCount = 0;

while (runCount < 3)

{

runCount++;

}
```

will run 3 times. In this code, runCount starts being equal to 0. The while loop starts, and the computer checks the value of runCount. 0 < 3, so it runs the code between the curly brackets. runCount increases by 1, and since the end of the code has been reached, the computer checks the condition again before restarting. 1 < 3, so it runs again. 2 < 3, so it runs once more after that. However, 3 < 3 is not correct, so the while loop ends and the script finishes.

For these sorts of numerical comparisons, you can use the following operators: > (greater than), < (less than), == (equal to), >= (greater than or equal to), <= (less than or equal to), and != (not equal to). **Remember to use**

```
while (playerHealth > 0 && enemyHealth > 0)
```

Do not run your code yet. We have one last thing to go over. The computer will *not* pause on its own when running a while loop. Our loop has no pauses in it and will not end by itself, so if we run it now, it will attempt to repeat that loop forever, which causes the program to freeze and eat up all of your processing power. In case you couldn't tell, *that's very bad.* We don't want that to happen, so just throw a Console.ReadLine(); in that loop somewhere to force it to pause and wait for your input to continue. We don't need to set a variable to it yet, so just put it in on its own. The computer will simply forget whatever the user inputs.

If you've actually managed to retain everything from this step, then great job! Conditions like these will come up a lot while coding. If you don't get it all now, then don't worry. It'll make more sense as you continue to code and use them.

Your code from this segment should look like the image above. You don't need the extra lines, but since the computer ignores empty lines when running code, it's smart to use them for organization.

two equals signs when comparing with equal to. If you use only one sign, the computer will think that you're trying to set the variable inside the condition, which it can't do for several reasons.

Now let's get back to our game. Go into the empty parentheses next to while in your script and add "playerHealth > 0" (without quotes) to it. Now that loop will run as long as the player has more than 0 health, but we're not done. We also want it to check that the enemy has more than 0 health. To include multiple conditions in one check, we can put both conditions next to each other and split them with either two ampersands (&&) to require both conditions be met or two vertical slashes (||) to require that only one of the conditions need to be met. Since we don't want the game to continue if one of the players has more than 0 health while the other doesn't, we use the ampersands. Add a check for enemyHealth being greater than 0 and then that line of code should look like below:

```
0 references
static void Main(string[] args)
{
    int playerHealth = 20;
    int enemyHealth = 20;

    while (playerHealth > 0 && enemyHealth > 0)
    {
        Console.WriteLine("The player has " + playerHealth + " health.");
        Console.WriteLine("The enemy has " + enemyHealth + " health.");

        Console.ReadLine();
    }
}
```

Step 6: Comments

After that last step is probably a good time to stop coding for a little and take some time to talk about comments. In most coding languages, you can write any kind of text within a script and have the computer ignore it by starting the text with some special characters. In C#, this is usually done with two slashes (//). Here is an example:

Console.WriteLine("This text will be shown in the console."); // This text will be ignored.

If the computer sees two slashes anywhere, it will ignore everything else in that line and skip straight to the next line of code. This lets you write text anywhere in the script without the program breaking.

Usually comments are used to leave little notes that help the developer remember what certain parts of the script do. While this doesn't directly change anything about what the user experiences, it helps immensely while making a program. Messy code is always bad code no matter how functional.

There's one more way to make a comment: if you use /* and */ instead of //, then the computer will ignore everything between the /* and */ rather than everything until the next line. You can use that to write a comment in the middle of a line of code or span a larger comment across multiple lines, like this:

Console.WriteLine("Hello world!" /*This text will be ignored, even though there's still more to this line of code.*/);

/*

This text will be ignored despite taking multiple lines.

*/

Be sure to mark the end of these comments, otherwise the script will break. I will be using comments in my main script going forward.

```
// The computer will ignore this!

/*
The computer will also ignore this!
*/

Console.WriteLine(/*The computer will even ignore this!*/"The computer will not ignore this.");
```

Step 7: If Statements

When I explained while loops, I combined it with the gameplay loop since they were both essentially the same thing. The next step uses *if* statements in addition to other things, so I'll explain if statements here since they're very important.

If statements check *if* a specific condition is true or false (correct or incorrect) and will run a specific set of code if it is true. The conditions used are the exact same as the ones used in the while loops. This example will write "testNumber is equal to 2!" in the console if testNumber is equal to 2:

```
int testNumber = 2;

if (testNumber == 2)

{

Console.WriteLine("testNumber is equal to 2!");
```

```
int test = 0;

if (test == 0)
{
    Console.WriteLine("test is equal to 0!");
}

test = 3;

if (test != 0)
{
    Console.WriteLine("test is not equal to 0!");
}
```

```
}
```

I suppose now is also a good time to say that you can also use == (equal to) and != (not equal to) to compare strings; this example will write "testString does not say test!" in the console if testString does not contain the text "test":

```
string testString = "test";

if (testString != "test")

{

print("testString does not say test!");

}
```

Step 8: Getting the Player's Command

Again, the gameplay for this game consists of the player saying whether they want to attack or defend. We need the game to first ask the player what they want to do, then run the correct code depending on the response. We'll first add Console.WriteLine(); within the gameplay loop (the while loop that contains everything) but before Console.ReadLine();. In it add some message explaining to the player that they should type "attack" if they would like to attack the enemy or "defend" if they would like to block the enemy's attack.

Now we want to assign that blank Console.ReadLine(); to a variable. Add string to the very start of the line with Console.ReadLine();, then make up a new name to describe the player's choice (such as choice or playerChoice) to write immediately after. To finish, follow it up with an equals sign. Doing it like this creates the variable with the reading of the player's input, which by extension makes it a part of the while loop. Due to how C# works, variables only exist to anything within the curly brackets containing the variable's declaration. This makes it slightly harder to develop at first, but it also makes the program run slightly better and organizes things. The reason we want the response to be recorded within the loop but not the health is because the response is only used within the loop and is set again every time the loop runs. The health is also only used within the loop, but it should remain the same value when the loop starts again as it was when the loop ended, thus we declare it outside of the loop so that the loop doesn't reset them by declaring them again.

After the code that reads the player's input, set two if statements and curly brackets one after another. One should check if the player's input reads "attack" and the other checks if the player's input reads "block". For now, just put a line of Console.WriteLine(); in each set of curly brackets with a message about the option that it's in. You should also add a line to the attacking section that removes 2 from enemyHealth, and if you want, add that you dealt 2 damage to the Console.WriteLine(); string too.

Now you can save and test your program. Depending on what you enter, it should either print your message about attacking, print your message about defending, or do nothing if you entered neither. If you have trouble, check the image above as a reference.

```
0 references
static void Main(string[] args)
{
    // Declare variables
    int playerHealth = 20;
    int enemyHealth = 20;

    while (playerHealth > 0 && enemyHealth > 0)
    {
        // Inform player of current game state
        Console.WriteLine("The player has " + playerHealth + " health.");
        Console.WriteLine("The enemy has " + enemyHealth + " health.");
        Console.WriteLine("Write 'attack' to attack the enemy or 'block' to block an enemy attack");

        // Get the player's choice
        string choice = Console.ReadLine();

        // Process player action
        if (choice == "attack")
        {
            // If the player attacks
            Console.WriteLine("You attacked for 2 damage!");
            enemyHealth -= 2;
        }
        if (choice == "block")
        {
            // If the player defends
            Console.WriteLine("You blocked against the attack!");
        }
    }
}
```

Step 9: Enemy's Turn

Now that we have the player's turn and all of the coding concepts down, we can program the enemy's turn. All it will do is deal a random amount of damage if the player hasn't chosen to block.

First we have to generate a random number to determine how much damage it will do, otherwise there would be no reason to block. We do this by using this line of code right before the if statements that process the player's action:

int enemyDamage = new Random().Next(1, 4);

This makes a new variable (enemyDamage) that will be something from 1-3. The reason it says 1 and 4 but produces 1-3 is because the second number is actually one above what the max number can be.

Now that we have the damage, we just plug it into the right spot. Right below where 2 is removed from the enemy's health add a line where enemyDamage is removed from the player's health. To finish it off, add a Console.WriteLine(); to both the if statements for the player attacking or blocking and have the console write how much damage the player took/would've taken.

Like all the steps before, and example is shown above in case you need help. Remember to save!

```
0 references
static void Main(string[] args)
{
    // Declare variables
    int playerHealth = 20;
    int enemyHealth = 20;

    while (playerHealth > 0 && enemyHealth > 0)
    {
        // Inform player of current game state
        Console.WriteLine("The player has " + playerHealth + " health.");
        Console.WriteLine("The enemy has " + enemyHealth + " health.");
        Console.WriteLine("Write 'attack' to attack the enemy or 'block' to block an enemy attack");

        // Get the player's choice
        string choice = Console.ReadLine();
        // Decide enemy damage
        int enemyDamage = new Random().Next(1, 4);

        // Process player action
        if (choice == "attack")
        {
            // If the player attacks
            Console.WriteLine("You attacked for 2 damage!");
            enemyHealth -= 2;
            Console.WriteLine("The enemy attacked for " + enemyDamage + " damage!");
            playerHealth -= enemyDamage;
        }
        if (choice == "block")
        {
            // If the player defends
            Console.WriteLine("You blocked against the attack!");
            Console.WriteLine("The enemy would've dealt " + enemyDamage + " damage!");
        }
    }
}
```

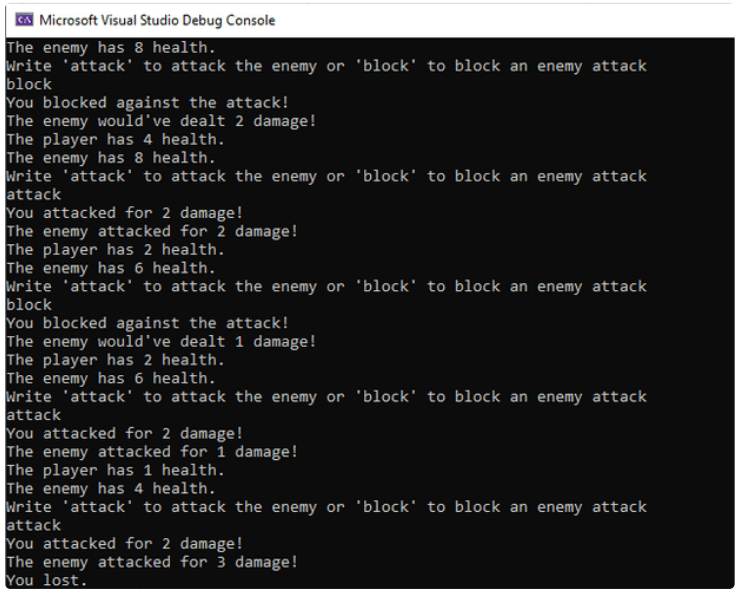
Step 10: Results

When the game ends, it's a good idea to tell the player who won. As it is, our game just ends abruptly once someone runs out of health.

The way that the while loop that contains our game works is that once it finishes running, the computer will continue to run all the code after the closed curly bracket. That code will only run once the loop has finished, so we'll put our ending there.

The ending itself consists of three if statements: one will check if just the player ran out of health (loss), one will check if just the enemy ran out of health (victory), and the last will check if both ran out of health (stalemate). Doing this should be pretty self-explanatory at this point. If you have trouble remember how, look earlier in your code and see how you did it then. I will not be providing an example of what this should look like because you should be able to figure it out yourself by now.

If you want to do more, I encourage you to try and take this idea further. Maybe add multiple attacks. Perhaps a way to heal. Any mechanic you can think of, you can probably do.

If you need more help learning to do more than what's on this guide, then I strongly encourage you to check out https://www.learncs.org/. It's a website that teaches C# programming in an interactive and intuitive way, much like this project. If there's still something you need to know beyond that or you want to know something specific to the .NET Core, then Google is your friend.

Thank you for reading and happy coding!